

Azure Data Factory (ADF)

Hands-On Notes

Azure Services Theory

Azure Blob Storage

- Access can be controlled using IAM roles.
- Data Storage is possible in the form of containers, file shares, queues and tables.
- Access to the account can be configured through Access Keys.
 - Two keys by default is generated.
 - The Key or the Connection String can be used to configure & establish connections from scripts or other tools.
- Desktop version is Azure Storage Explorer and is useful in case we do more work on the accounts.

Azure Data Lake Gen2 Storage

- Similar to Azure Blob Storage but it has hierarchical namespace enabled. During account creation, the options are similar but we have to enable the hierarchical namespace by ticking that box.

Azure SQL Database

- The pricing tier can be changed post-creation.
- During creation, ensure to mark SQL Authentication to the server with username and password to conveniently login to Query Editor.
- Query Editor can be used to run queries in the Web Console.
 - Azure Data Studio is the light-weight Desktop version that can be used for querying (engineering) purposes. It has no admin controls.
 - SSMS is the heavy-weight desktop version required for admin activities as well.

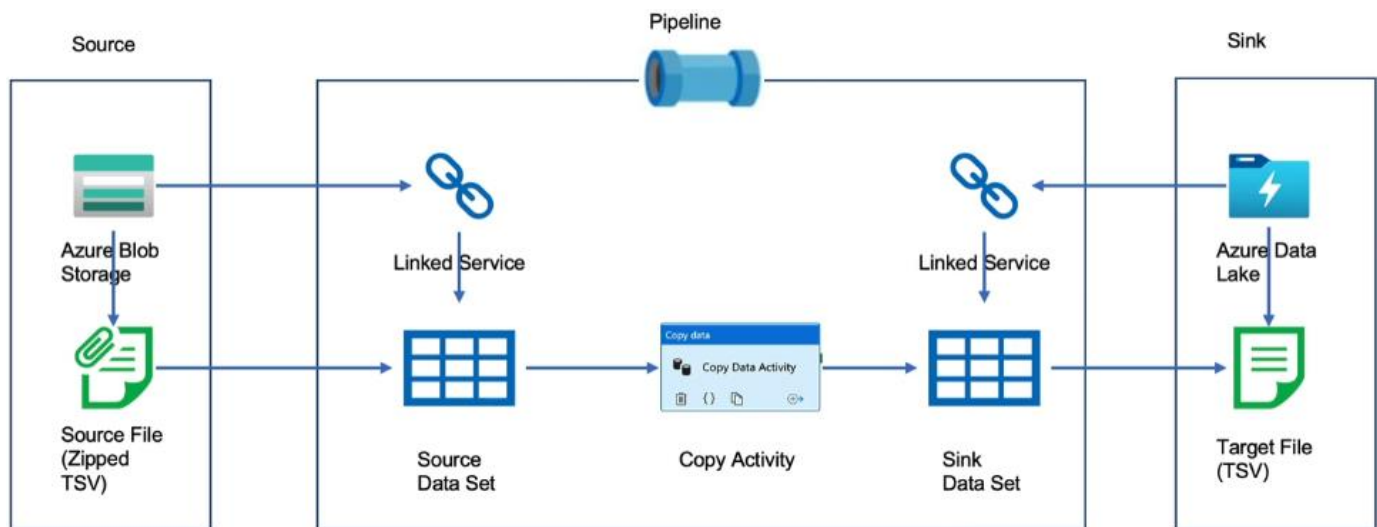
Azure Data Factory & Informatica

- Data Factories are like Repositories in Informatica
- Data Flows are like Mappings in Informatica
- Pipelines are like Sessions in Informatica
- Activities are like Transformations in Informatica
- Linked Services are like Connections in Informatica

- Datasets are like Source/Target Definitions in Informatica
- Triggers are like Scheduler in Informatica

Mapping 1a: Copy File from Blob to Data Lake

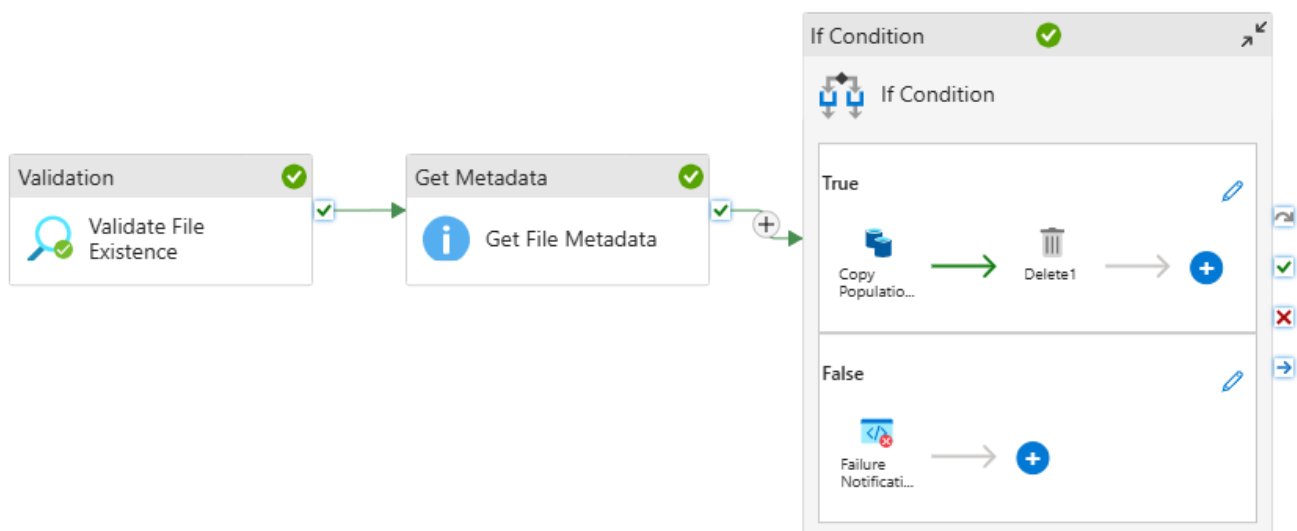
- Create: Copy Activity, Linked Services, Datasets, Pipeline
- Activities: Validation, If Condition, Web, Get Metadata, Delete
- Triggers for scheduling automated workflow



Storage Account: covidreportingsa
 Container: population
 File: population_by_age.tsv.gz

Storage Account: covidreportingdl
 Container: raw
 File: population/population_by_age.tsv

Mapping 1b: Enhancing 1a to Trigger based on File Availability & Accuracy



Additional Activities Used

Validation Activity: The Validation activity in Azure Data Factory ensures that data meets specific criteria before proceeding in a pipeline. It checks for the existence of datasets, validates their size, and can confirm the presence of child items in folders.

Get Metadata Activity: The Get Metadata activity in Azure Data Factory is used to retrieve metadata from a dataset or file. It allows you to extract details such as file name, file size, item count, last modified date, and more. This activity is commonly used in data pipelines for conditional logic, such as dynamically determining further steps based on file or folder properties.

If Condition Activity: The **If Condition activity** in Azure Data Factory enables conditional branching within a pipeline. It evaluates a specified expression and determines whether it resolves to *true* or *false*. Based on the result:

- If **true**, the pipeline executes the activities listed under the "If True" branch.
- If **false**, the pipeline executes the activities under the "If False" branch.

This activity is particularly useful for creating dynamic workflows, where subsequent steps depend on runtime conditions or data states. It allows for greater flexibility and decision-making in pipeline logic.

One notable feature of this is that the True/False categories can have their own list/sequence of activities that are hidden from the main pipeline page. It's like a worklet of activities.

Triggers

There are 4 types of triggers in ADF – Schedule Trigger, Tumbling Window Trigger, Storage Events Trigger & Custom Event Trigger.

Schedule Trigger

- Runs on a calendar/clock
- Supports periodic & specific times
- Trigger to pipeline is many-to-many
- Can only be scheduled for a future time to start

Tumbling Window Trigger

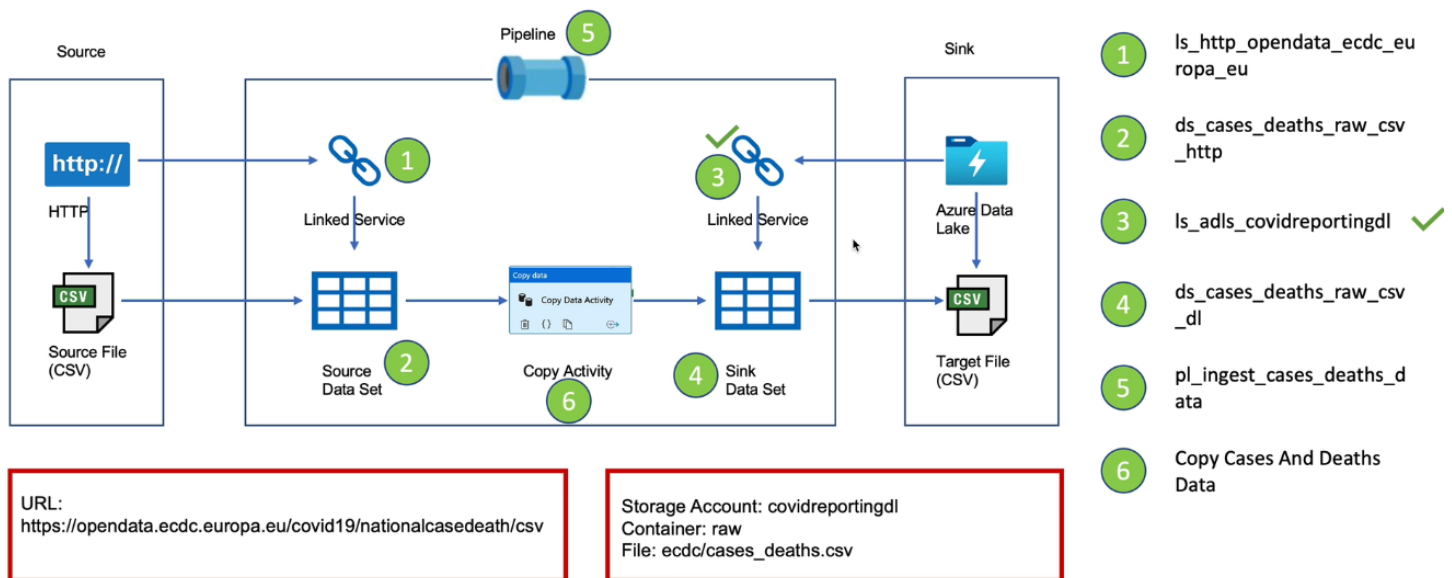
- Runs at periodic intervals
- Windows are fixed-sized & non-overlapping
- Can be scheduled for past windows/slices
- Trigger to pipeline is one-to-one

Storage Events Trigger

- Runs in response to Storage Events
- Events can be creation or deletion of Blobs/files
- Trigger to pipeline is many-to-many

Mapping 2a: Simple HTTP File Copy to ADLS

Copy Activity – Case & Deaths Data



This is similar to pipeline in #1a. Steps are as below:

- Create a linked service for the HTTP connection.
- Create/Reuse linked service for ADLS.
- Create a dataset for source & target respectively.
- Create a pipeline for copying data.

So we create/need ~4 objects to load one file from a website/location.

In this approach, the problem is that if we have 10 files to pull from one website, we will have to create ~400 objects to load data. Thus, we explore parameterization.

Mapping 2b: Metadata-Driven HTTP File Copy to ADLS

This mapping is to explore the settings and usage of parameters & variables in ADF.

Setting 1: Exploring the Usage of Parameters & Variables

Let's create a new pipeline (pl_ingest_hospitalisation_data_using_variables). Let's also create a new dataset for both source (ds_ecdc_data_raw_http) & target (ds_ecdc_data_raw_dl) data.

Let's now tweak the parameter & variable values. Below are the steps:

- Add relativeURL parameter to source dataset.
- Add filename parameter to target dataset.
- Add variables (sourceRelativeURL & sinkFileName) in pipeline.
- Assign the pipeline variables values as the dataset parameter values in the Copy Activity.

Note: The relativeURL & filename values can be given as default value in the variables (in pipeline) or the parameters (in datasets).

Now, let's incorporate triggers into our mapping in the next setting.

Setting 2: Incorporating Triggers to Pass Values During Runtime

In Setting #1, we used variables in the pipelines. Now, let's switch that to parameters first.

Let's create a new pipeline (pl_ingest_testing_data_with_runtime_parameters); only difference from setting #1 is that we create parameters in the pipeline instead of variables.

But here, we still have to set the values for the parameters manually during runtime.

Data Flows

Overview

- These are like mappings in Informatica where we can do code-free transformations.
- The Data Flows need to be included in the Pipelines (akin to infa sessions) to trigger them.
- Data Flows will get submitted to a Databricks Cluster. Thus, an appropriate cluster needs to exist to run these jobs.
- For testing purposes, we can also enable debug settings in the Data Flows so as to be able to preview the results of the transformations in each step. It works like the debugger setting in Informatica.
- The source and sink are mandatory transformations for every data flow.

Source Transformation

We can pick a dataset or inline option as the source transformation:

- **Dataset:** These are reusable entities that define the schema and location of your data. They are ideal when you need a consistent schema across multiple pipelines or activities, such as Copy Data. Datasets are not based on Spark and are useful when working with a hardened schema2.
- **Inline:** These are defined directly within the source transformation and are recommended for flexible schemas, one-off source instances, or parameterized sources. Inline datasets are based

on Spark and allow you to avoid creating additional dataset objects, making them more dynamic and suitable for heavily parameterized operations².

If you need reusability and schema consistency, go for Datasets. For flexibility and simplicity in specific scenarios, Inline is the way to go.

Few other things to note:

- If the schema has not been defined for the dataset used, then *Projection -> Import Projection* option can be used to get the schema inferred by ADF.
- If we don't want to run the debug on the full file, we have 2 options in the *Debug Settings*:
 - Upload a sample file with less data and have the debug run on this sample file. I believe this will help with testing edge cases and general unit testing effort to a great extent.
 - Limit the number of records being sampled by assigning a sample size (default is 1000) in debug settings.

Filter Transformation

- We can give the filter condition in the filter settings and get just the filtered data to flow to the rest of the pipeline.
- Data Preview can be used to visualize results inside the filter condition editor as well as outside it.

Note >> The condition in the filter clause is used to PICK records that meet the condition and NOT to eliminate. I.e., if a record matches the condition we set in the filter, it WILL PASS through.

Select Transformation

Select transformation in ADF is similar to an Expression transformation in Informatica. As such, there is a functionality to create Rule-based columns in ADF similar to how we create additional fields in Informatica. Just like the latter, names of columns can also be easily changed.

The columns that satisfy the rules will show up under the Rule-based mapping as shown below:

✓ Validate [Debug Settings](#)

Select settings Optimize Inspect Data preview ● [Description](#)

Output stream name * [Learn more](#)

Incoming stream *

Options

- ☐ Skip duplicate input columns ⓘ
- ☐ Skip duplicate output columns ⓘ

Input columns * ☒ Auto mapping ⓘ [Reset](#) [+ Add mapping](#) [Delete](#) 1 mappings: 4 column(s) from the inputs left unmapped ⓘ

☐ FilterEuropeOnly's column	Name as
☒ type == 'string'	\$\$ + '_string' abc
☐ country	country_string
☐ country_code	country_code_string
☐ continent	continent_string
☐ indicator	indicator_string
☐ source	source_string

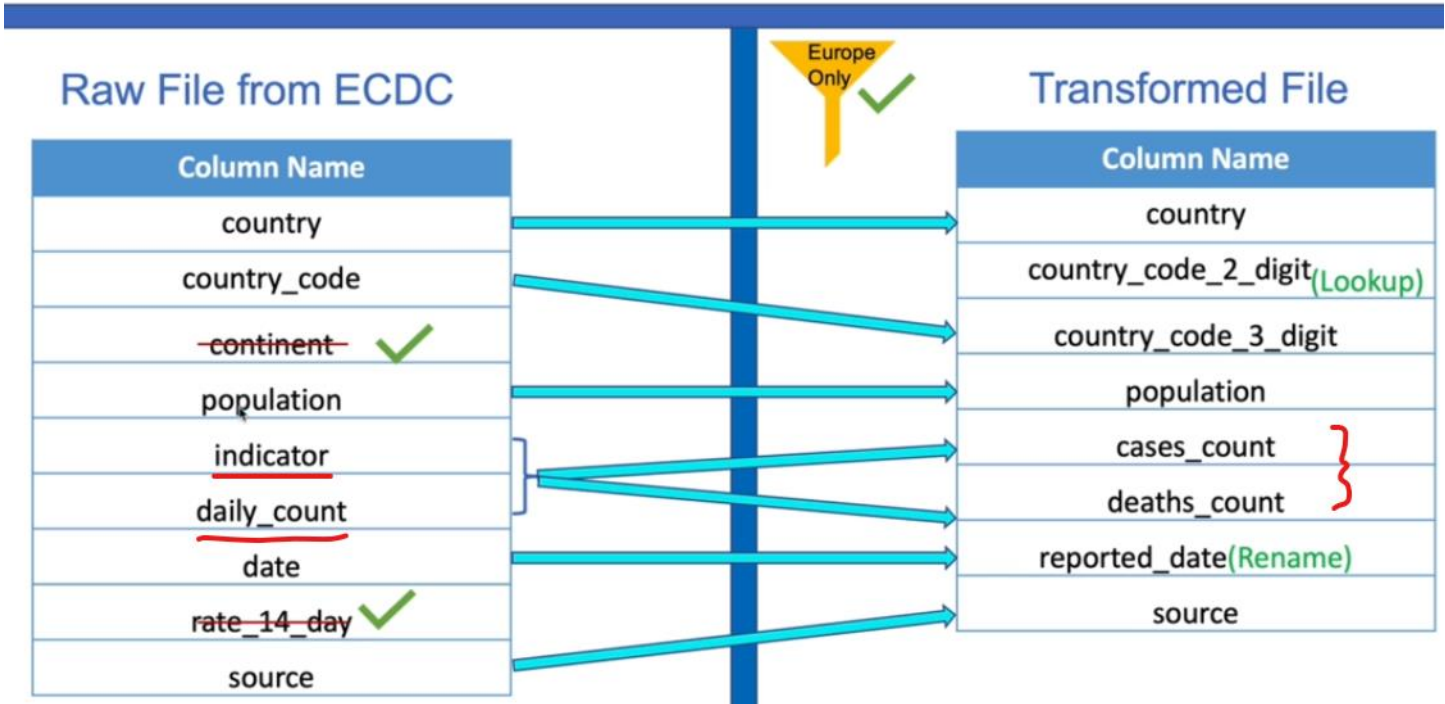
The above screenshot suffixes `'_string'` to all columns of datatype='string'.

Pivot Transformation

This helps implement the SQL Pivot operation on the incoming dataset.

Let's look at how to implement this. Consider the below requirement in red.

Transform Cases & Deaths Data



We want to create the columns cases_count & deaths_count using the indicator & daily_count columns from source.

There are 3 tabs – Group By, Pivot key, Pivoted columns.

Group by does what it says. But unlike a SQL query, if no column is specified, it will just give a single record. I.e. it aggregates by null. Note that we can create new user-defined columns here as well.

1. Group by		2. Pivot key	3. Pivoted columns
Columns		Name as	
abc	country	country	+ 🗑
abc	country_code	country_code	+ 🗑
123	population	population	+ 🗑
📅	reported_date	reported_date	+ 🗑
abc	source	source	+ 🗑

The screenshot above has a new, user-defined column country_special that concatenates the 2 columns – country & country_code.

Pivot Key is the column based on which we want to pivot.

Incoming stream * SelectOnlyRequiredFields

1. Group by 2. **Pivot key** 3. Pivoted columns

Pivot key * abc indicator

Value

+ 🗑️

☐ Null value

Here, we can see that the indicator column is the Pivot key. Now, ***even if we don't give the values, ADF will fetch that during run time. Thus, values are optional.***

Pivoted Columns tab is where we can customize the operation to be done, the names of the resulting columns and their positioning among other columns.

1. Group by 2. Pivot key 3. **Pivoted columns**

Column name pattern * prefix{expression prefix}middle{Pivot key value}suffix

Prefix Middle Suffix

1. Group by 2. Pivot key 3. **Pivoted columns**

Column name pattern * prefix{expression prefix}middle{Pivot key value}suffix

Prefix prefix{expression prefix}middle{Pivot key value}suffix prefix{Pivot key value}middle{expression prefix}suffix

The above shows how we can decide prefix | middle | suffix patterns.

The expression can be given in the pivot expression boxes. Overall, it looks like this.

Incoming stream * SelectOnlyRequiredFields

1. Group by 2. Pivot key 3. **Pivoted columns**

Column name pattern * prefix{Pivot key value}middle{expression prefix}suffix

Prefix - Suffix

Column arrangement * Normal **Lateral**

sum(daily_count) 121 count + 🗑️

- *count* is the expression name and *sum(daily_count)* is the expression formula.

- Result will look like this.

Derived Column Transformation

We can work around this by creating a Derived Column Transformation that will ensure the mapping of the new columns.

Lookup Transformation

But there are some fundamental changes in how it is created. In order to use the lookup, we must first add the dataset on which we need to do a lookup on in the data flow as a source transformation in a

secondary stream. Then, we can create a lookup to the primary stream and connect it to the secondary stream. It works like a left outer join where the primary stream is the master.



When you look at it that way, it works more like a joiner in Informatica than a lookup. However, it ADF lookup also has the functionality to decide what to do on multiple matches just like Informatica lookup.

For our requirement, the **Lookup settings** tab will look like this.

Lookup settings	Optimize	Inspect	Data preview	●
Output stream name *	LookupCountry			Learn more
Description	Lookup on 'PivotCounts' from 'CountryLookup'			Reset
Primary stream *	PivotCounts			
Lookup stream *	CountryLookup			
Match multiple rows	☐ ⓘ			
Match on *	Any row			
Lookup conditions *	Left: PivotCounts's column			Right: CountryLookup's column
	abc country	==	abc country	+ 🗑️

In the Optimize tab, there are optimization options that are suited for PySpark clusters.

Lookup settings	Optimize	Inspect	Data preview	●
Broadcast	☒ Auto ⓘ ☐ Fixed ⓘ ☐ Off ⓘ			
Partition option *	☒ Use current partitioning ☐ Single partition ☐ Set partitioning			

Sink Transformation

The Sink Transformation in ADF is equivalent to the Target Definition in Informatica.

The screenshot shows the 'Sink' tab of the Sink Transformation configuration in Azure Data Factory. The interface includes a top navigation bar with tabs: Sink, Settings, Errors, Mapping, Optimize, Inspect, and Data preview. The main configuration area contains the following fields and options:

- Output stream name ***: A text box containing 'CasesAndDeathsSink'. A 'Learn more' link is to the right.
- Description**: A text box containing 'Export data to ds_cases_and_deaths_processed_csv_dl'. A 'Reset' button is to the right.
- Incoming stream ***: A dropdown menu showing 'SelectForSink'.
- Sink type ***: Three buttons: 'Dataset' (selected), 'Inline', and 'Cache'.
- Dataset ***: A text box containing 'ds_cases_and_deaths_processed_csv...'. To the right are links for 'Test connection', 'Open', and 'New'. Above this section is a green checkmark and the text 'Connection successful'.
- Skip line count**: An empty text box.
- Options**: Two checkboxes: 'Allow schema drift' (checked) and 'Validate schema' (unchecked).

Other advanced options can be explored in other tabs as per requirement. Just remember that it is a Spark Cluster that stores data. File structures and organization will be accordingly.

Conditional Split Transformation

Split Column is used to break a single column into multiple columns based on a delimiter or fixed character length.

Key Features:

- Split by delimiter (e.g., comma ,, semicolon ;, space, etc.).
- Split by fixed number of characters (e.g., after every 5 characters).
- Create new columns dynamically as output for each split part.
- Helps when dealing with concatenated string fields (like CSV fields inside a column).

Common Use Cases:

- Splitting a full name column into First Name and Last Name.
- Splitting a CSV-style string into multiple structured columns.
- Breaking down codes or IDs embedded inside a text field.

Conditional split settings Optimize Inspect Data preview

Output stream name *

SplitDailyAndWeekly

Learn more

Description

Conditionally distributing the data in indicator groups, based on columns '{1}'

Reset

Incoming stream *

RemoveDuplicates

Split on

☒ First matching condition ☐ All matching conditions

Split condition

Stream names	Condition	
Daily	trim(indicator) == 'Daily hospital occupancy' trim(indicator) == 'D...' %	+
Weekly	Rows that do not meet any condition will use this output stream	+

Aggregate Transformation

Aggregate transformation is used to group rows together and apply aggregate functions like sum, count, average, min, max, and more.

It works similarly to a SQL GROUP BY operation but happens visually inside the Data Flow.

Key Features:

- Group rows based on one or more columns.
- Apply multiple aggregate functions (SUM, AVG, COUNT, MIN, MAX, etc.) during the grouping.
- Perform complex groupings with expressions and conditional aggregates.
- Outputs one row per group.

Common Use Cases:

- Summarizing sales totals by region or by month.
- Counting the number of orders per customer.
- Finding the maximum value per category.
- Calculating average scores by group.

How it Works:

- Group By: Select one or more columns on which to group data.
- Aggregates: Choose aggregation functions (SUM, COUNT, AVG, etc.) and specify the target columns.

Aggregate settings

Optimize

Inspect

Data preview

Output stream name *

GetWeekStartAndEndDates

[Learn more](#)

Description

Aggregating data by 'ecdc_year_week' producing columns 'Start_Date, End_Date'

Reset

Incoming stream *

AddYearOfWeekColumn

Group by

Aggregates

Columns

Name as

abc ecdc_year_week

ecdc_year_week

+

Join Transformation

Join transformation is used to combine rows from two different streams based on a matching condition. It works similarly to SQL joins (INNER JOIN, LEFT OUTER JOIN, etc.) inside a Data Flow.

Key Features:

- Supports multiple join types: Inner, Left Outer, Right Outer, Full Outer, Left Anti, Right Anti.
- Allows joining based on one or more conditions.
- Handles complex expressions in join conditions (e.g., calculated fields).
- Supports joining on different column names between the two datasets.

Common Use Cases:

- Enriching a dataset by joining master data (like customer details to transactions).
- Filtering records based on existence in another dataset (using Anti Joins).
- Merging datasets from different sources into a unified structure.
- Combining historical and live data streams for analysis.

How it Works:

- Left Stream: The first dataset (primary).
- Right Stream: The second dataset (to join with left).
- Join Condition: The logic on which rows from left and right are matched.
- Join Type: Defines how unmatched rows are handled (Inner, Left Outer, etc.).

Sort Transformation

Sort transformation is used to order rows in your data based on one or more columns, either in ascending or descending order.

It helps when the output needs to be processed or consumed in a particular order, such as for ranking, windowing, or preparation before writing to destinations that require sorted data.

Key Features:

- Sort by one or multiple columns.
- Set ascending or descending order individually for each column.
- Supports case sensitivity options for text-based sorting.
- Can be used before transformations like Rank, Window, or Aggregate where order matters.

Common Use Cases:

- Sorting sales records by amount or date before further processing.
- Preparing ranked lists (e.g., top 10 customers, highest scores).
- Sorting before using Window functions (like running totals, lead/lag).
- Ensuring deterministic outputs when writing data to ordered formats like parquet partitions.

How it Works:

- Column: Choose the column(s) to sort on.
- Sort Order: Choose Ascending or Descending for each column.
- Case Sensitivity (Optional): specify if sort should consider character casing.

Azure HDInsight

Azure HDInsight is like databricks for multiple big-data tools like Hive, Spark, Kafka, etc.

What Exactly is Azure HDInsight?

At its core, Azure HDInsight is a managed cloud service that provides an environment to deploy, run, and scale open-source frameworks and tools for big data analytics. Think of it as the underlying platform that abstracts away the complexity of setting up and managing infrastructure for these frameworks.

What Azure HDInsight Does

Infrastructure Management: HDInsight provisions the necessary compute and storage resources (virtual machines, networking, and distributed storage systems like Azure Blob Storage) required to run big data frameworks like Apache Spark, Hive, Hadoop, Kafka, etc. You don't have to manually set up or manage servers or clusters.

Framework Configuration: HDInsight comes pre-configured with popular big data frameworks. For example:

- Spark as a distributed data processing engine.
- Hive to run SQL-like queries on distributed data stored in a Hadoop-based file system (HDFS).
- Kafka for processing real-time streaming data. By leveraging HDInsight, you essentially get these tools ready-to-go, without needing to manually install or configure them.

Integration: It integrates seamlessly with other Azure services like Azure Data Lake Storage, Azure Active Directory (for access control and security), and Azure Monitor (for monitoring and managing your clusters). This integration simplifies workflows.

How Azure HDInsight Run Frameworks?

Azure HDInsight acts as a containerized service to host these frameworks. Instead of requiring users to set up on-premises clusters or virtual machine instances for their tools:

- It spins up clusters dynamically based on the framework/tool you choose, so the frameworks run "on top" of the HDInsight-managed environment.
- HDInsight ensures scalability (adding or removing nodes in the cluster), availability, and performance monitoring while running your workloads.

In short, HDInsight serves as a middleman that lets you focus on using tools like Spark, Hive, or Kafka for their intended purposes without worrying about the nitty-gritty of setting up distributed systems or managing clusters.

Azure Databricks

Azure Databricks is just enabling Databricks inside the Azure portal.

Orchestration

If we think about it generally, the below is how we see the pipeline orchestration requirements:

- Pipeline executions need to be fully automated.
- Pipelines run at regular intervals or on an event occurring.
- Activities only run once the upstream dependency has been satisfied.
- Easier to monitor for execution progress and issues.

Azure Data Factory (ADF) has the below capabilities:

- Dependency between activities inside a pipeline.
- Dependency between pipelines within a parent pipeline.
- Dependency between triggers; ONLY for tumbling window triggers.
- Create custom-made solution via Power Shell, SDKs, Python, Rest APIs. Last resort as this incurs additional cost and adds to complexity.

Execute Activity

The **Execute Pipeline Activity** allows you to invoke one pipeline from another pipeline within Azure Data Factory or Azure Synapse Analytics.

Key Features

1. **Modularity:** Breaks complex ETL processes into manageable components for easier development and maintenance.
2. **Dependency Management:** Ensures data workflows run in the correct sequence, reducing errors and improving data quality.
3. **Flexibility:** Allows passing parameters and customizing child pipelines to adapt to changing data requirements.

Configuration Steps

1. **Add Activity:** Drag the Execute Pipeline activity to the pipeline canvas.
2. **Select Pipeline:** Choose an existing pipeline or create a new one.
3. **Configure Parameters:** Set any required parameters for the invoked pipeline.
4. **Wait on Completion:** Optionally, configure whether the activity waits for the invoked pipeline to complete.

Example Use Case

- **Master Pipeline:** Contains an Execute Pipeline activity that calls an invoked pipeline.
- **Invoked Pipeline:** Performs a specific task, such as copying data from one source to another.

Option #1: Dependencies Using Execute Activity

For this, we need just 2 additional steps provided we already have the pipelines in place:

- Create a new pipeline for orchestration. Use the Execute activity in this pipeline to create the desired run sequence.



- Create a trigger to run the orchestration pipeline.

Name *

Description

Type *

BlobEventsTrigger

Account selection method * ⓘ

☒ From Azure subscription ☐ Enter manually

Trigger name ↑↓	Trigger type	Trigger time ↑↓	Status ↑↓	Pipelines	Run
tr_population_file_arrived	Storage events trigger	4/29/2025, 4:40:29 PM	✓ Succeeded	1	Original

☐ Pipeline name ↑↓	Run start ↑↓	Run end ↑↓	Duration	Triggered by	Status ↑↓	Run
☐ pl_transform_population_data	4/29/2025, 4:41:16 PM	4/29/2025, 4:47:30 PM	6m 14s	2afc86ec-bad5-48a4...	✓ Succeeded	Original
☐ pl_ingest_population_data	4/29/2025, 4:40:33 PM	4/29/2025, 4:41:15 PM	42s	c7ea3c47-ddbc-4cff...	✓ Succeeded	Original
☐ pl_execute_population_pipelines	4/29/2025, 4:40:30 PM	4/29/2025, 4:47:32 PM	7m 3s	tr_population_file_ar...	✓ Succeeded	Original

Tumbling Window Triggers

Tumbling window triggers in Azure Data Factory are a type of trigger that fires at periodic time intervals from a specified start time. These intervals are fixed-sized, non-overlapping, and contiguous.

Key Concepts

1. Tumbling Window:

- **Fixed-sized intervals:** Each window is of a fixed duration (e.g., 1 hour, 1 day).
- **Non-overlapping:** Windows do not overlap; each window starts immediately after the previous one ends.
- **Contiguous:** Windows are continuous without gaps.

2. Trigger Properties:

- **Frequency:** Defines how often the trigger fires (e.g., every hour).
- **Interval:** The duration of each window.
- **Start Time:** When the first window starts.
- **End Time:** Optional, defines when the last window ends.
- **Max Concurrency:** Maximum number of concurrent pipeline runs.
- **Retry Policy:** Defines how many times and at what interval the trigger should retry in case of failure.

Tumbling Window Trigger Workflow

1. Trigger Initialization:

- The trigger starts at the specified start time.
- Each window is created based on the defined interval.

2. Pipeline Execution:

- For each window, the associated pipeline is executed.
- If a pipeline fails, the retry policy determines the retry behavior.

3. Dependency Management:

- Tumbling window triggers can depend on other triggers.
- Self-dependency ensures that the next window does not start until the current window's pipeline execution is complete.

Option #2: Dependencies Using Tumbling Window Triggers

Name *
tr_process_cases_and_deaths_data

Description

Type *
TumblingWindowTrigger

Start Date (UTC) *
4/28/2025, 12:00:00 AM

Recurrence * ⓘ
Every 24 Minute(s) ▼

☒ Specify an end date

End On (UTC) * ⓘ
4/30/2025, 3:41:00 PM

▼ Advanced

Add dependencies
+ New | Delete

☐ Trigger	Offset	Window size
☐ tr_ingest_ecd... ▼	0.00:00:00	0.00:00:00

Delay ⓘ
00:00:00

Max concurrency * ⓘ
1

Monitoring

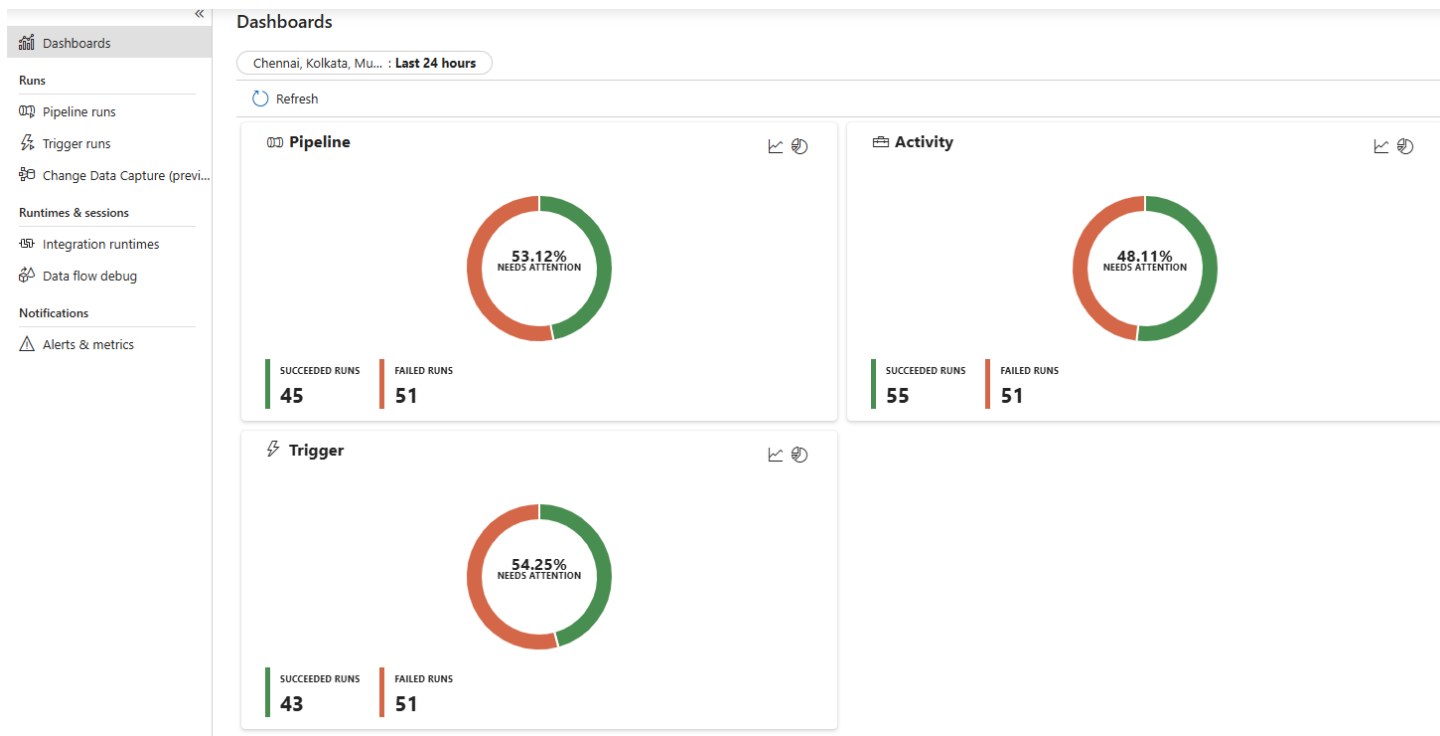
ADF is a managed service running on a serverless architecture. Thus, the monitoring is mostly about the jobs involved and not much on the infra side.

The general monitored aspects in ADF are:

- Azure Data Factory Service: Size of our ADF, Count of objects created, etc.
- Integration Runtime: CPU Utilization, Available memory
- Status of Trigger runs
- Status of Pipeline runs
- Status of Activity runs

Data Factory Monitor

Dashboard



Out-of-the-box; cannot be customized.

Pipeline Runs

Pipeline runs										
Triggered Debug Rerun Cancel options Refresh Edit columns List Gantt										
Filter by run ID or name Chennai, Kolkata, Mu... : Last 24 hours Pipeline name : All Status : All Runs : Latest runs Triggered by : All Add filter Copy filters Export to CSV										
Showing 1 - 100 items										
☐	Pipeline name	Run start	Run end	Duration	Triggered by	Status	Run	Parameters	Annotations	Run ID
☐	pl_load_cases_deaths_data	4/29/2025, 9:35:10 PM	4/29/2025, 9:35:30 PM	20s	tr_load_cases_and_d...	Succeeded	Original			19ad4051-f21c-44b3-9fae-2da...
☐	pl_load_hosp_admissions_dly_data	4/29/2025, 9:34:56 PM	4/29/2025, 9:35:23 PM	27s	tr_load_hosp_admiss...	Succeeded	Original			2d92e63d-150d-4eaf-977d-1b5
☐	pl_load_cases_deaths_data	4/29/2025, 9:34:53 PM	4/29/2025, 9:35:14 PM	22s	tr_load_cases_and_d...	Succeeded	Original			4a82ff84-eada-44af-a411-212e
☐	pl_load_hosp_admissions_dly_data	4/29/2025, 9:34:42 PM	4/29/2025, 9:35:01 PM	19s	tr_load_hosp_admiss...	Succeeded	Original			1928038a-e6fe-4faf-bea7-42d8
☐	pl_load_cases_deaths_data	4/29/2025, 9:34:38 PM	4/29/2025, 9:34:56 PM	18s	tr_load_cases_and_d...	Succeeded	Original			d946e641-ff00-449d-act7-39b5
☐	pl_load_hosp_admissions_dly_data	4/29/2025, 9:34:26 PM	4/29/2025, 9:34:51 PM	25s	tr_load_hosp_admiss...	Succeeded	Original			9b368b1b-0293-40ab-9565-86f

If we click through, we can see the Activity runs.

Home

Dashboards

Runs

Pipeline runs

Trigger runs

Change Data Capture (previ...

Runtimes & sessions

Integration runtimes

Data flow debug

Notifications

Alerts & metrics

All pipeline runs > pl_transform_cases_deaths_data - Activity runs

Rerun

Cancel

Refresh

Update pipeline

List

Gantt

Data flow

TransformCasesAndDeaths

Activity runs

Pipeline run ID 062256e1-fd94-4933-9b82-eb0e29fc93a6

All status

Showing 1 - 1 items

Activity name	Activity stat...	Activity ...	Run start	Duration	Integration runtime	User proper...	Activity run ID	Log
TransformCasesAndDeaths	Succeeded	Data flow	4/29/2025, 9:24:47 PM	5m 29s	AutoResolveIntegrationRuntime (East US)		b8e48a1d-ebd7-496e-8d8b-232a985c5e5d	

If we click through further, we can see the dataflow stage details.

All pipeline runs > pl_transform_cases_deaths_data - Activity runs > TransformCasesAndDeaths

TransformCasesAndDeaths

Cluster startup time: 4m 8s Number of transformations: 9 Data flow status: Success

Refresh

Auto refresh

On

Edit dataflow

Sinks All streams

Sink	Status	Processing time	Highest processing time	Rows written	Stages	Lineage
CasesAndDeathsSink	Succeeded	15s	5s	596		

Stages

Transformations

Stages

CasesAndDeathsSink

Processing time: 15s

TRANSFORM	ROWS	TIME
SelectOnlyRequiredFields	596	
FilterOnlyAsia	596	
CasesAndDeathsSource	41273	4s 187ms
CountryLookup	208	469ms
MapDerivedColumns	596	
PivotCounts	596	145ms
CasesAndDeathsSink	596	
SelectForSink	596	
LookupCountry	596	5s

CasesAndDeathsSink

Processing time: 15s

STAGE ID	ROWS READ	ROWS WRITTEN	TIME
4	129270	4	4s 187ms
13	208	0	469ms
14	0	0	2s 91ms
19	596	596	145ms
23	596	596	5s

Lineage

CasesAndDeathsSink

COLUMN	METHOD	ORIGINAL SOURCE
reported_date	Mapped	CasesAndDeathsSource(date)
deaths_count	Calculated	MapDerivedColumns(deaths_count)
source	Mapped	CasesAndDeathsSource(source)
country_code_3_digit	Mapped	CasesAndDeathsSource(country_code)
country_code_2_digit	Mapped	CountryLookup(country_code_2_digit)
confirmed cases_count	Calculated	MapDerivedColumns(confirmed cases_count)
population	Mapped	CasesAndDeathsSource(population)
country	Mapped	CasesAndDeathsSource(country)
-	Used	CountryLookup(country), CasesAndDeathsSource(country, country_code, continent)

Trigger Runs

»

«

Home

Runs

Pipeline runs

Trigger runs

Change Data Capture (previ...

Runtimes & sessions

Integration runtimes

Data flow debug

Notifications

Alerts & metrics

Trigger runs

AllScheduleTumbling windowStorage eventsCustom eventsRefreshEdit columns

Chennai, Kolkata, Mu... : Last 24 hoursTrigger name : AllStatus : AllRuns : Latest runsExport to CSV

Showing 1 - 100 items

Trigger name	Trigger type	Trigger time	Status	Pipelines	Run	Message	Properties	Run ID
tr_load_cases_and_deaths...	Tumbling window trigger	4/29/2025, 9:23:58 PM	Canceled	0	Original			085845566484688962397...
tr_load_cases_and_deaths...	Tumbling window trigger	4/29/2025, 9:23:58 PM	Canceled	0	Original			085845566484701874823...
tr_load_hosp_admissions...	Tumbling window trigger	4/29/2025, 9:23:55 PM	Canceled	0	Original			085845566484983318426...
tr_load_cases_and_deaths...	Tumbling window trigger	4/29/2025, 9:23:58 PM	Canceled	0	Original			085845566484721090056...
tr_load_hosp_admissions...	Tumbling window trigger	4/29/2025, 9:23:55 PM	Canceled	0	Original			085845566484991173809...

Integration Runtimes

»

«

Dashboards

Runs

Pipeline runs

Trigger runs

Change Data Capture (previ...

Runtimes & sessions

Integration runtimes

Data flow debug

Notifications

Alerts & metrics

Integration runtimes

AllSelf-HostedAzureAzure-SSISAirflow

Refresh

Edit columns

Time zone : Chennai, Kolkata, Mumbai, New ...

Showing 1 - 1 of 1 items

Name ↑↓	Type ↑↓	Sub-type ↑↓	Status ↑↓	Region ↑↓	Created on ↑↓
AutoResolveIntegrationRuntime	Azure	Public	Running	Auto Resolve	---

Alerts & Metrics

Alerts can be set-up in the monitor based on the metrics

Add criteria

Select one metric to set up the alert condition.

Metrics ↑↓

Failed SSIS integration runtime start metrics

Failed SSIS package execution metrics

Failed trigger runs metrics

Integration runtime available memory

»

«

Dashboards

Runs

Pipeline runs

Trigger runs

Change Data Capture (previ...

Runtimes & sessions

Integration runtimes

Data flow debug

Notifications

Alerts & metrics

Alerts & metrics

RefreshMetricsNew alert rule

ALERT	ENABLED	RESOURCE TYPE	RESOURCES	ACTIONS
al_ecdc_trigger_failure	Off	Trigger	1	

Failure Email

☑ Focused Other Filter

Microsoft Azure
Azure: Activated Severity: 0... 14:50
al_trigger_failure was triggered for covid-re...

Azure: Activated Severity: 0 al_trigger_failure

Microsoft Azure <azure-noreply@microsoft.com>
Mon 30/11/2020 14:50
To: You

Microsoft Azure

⚠ Your Azure Monitor alert was triggered

Azure monitor alert rule al_trigger_failure was triggered for covid-reporting-adf at November 30, 2020 14:50 UTC.

Rule ID	/subscriptions/34cf7d7a-1340-43c5-81e4-57efa1d4b11a/resourcegroups/covid-reporting-rg/providers/Microsoft.Insights/metricalerts/al_trigger_failure View Rule >
Resource ID	/subscriptions/34cf7d7a-1340-43c5-81e4-57efa1d4b11a/resourcegroups/covid-reporting-rg/providers/Microsoft.DataFactory/factories/covid-reporting-adf View Resource >

Rerun Capability

Fix the issue and then rerun the whole job or from the failed activity.

All pipeline runs > pl_ingest_ecdc_data - Activity runs

pl_ingest_ecdc_data

List Gantt

Rerun Rerun from activity Rerun from failed activity Refresh

Lookup Lookup ECDC File List

ForEach Execute Copy For Every Record

Activity runs

Pipeline run ID 401202de-3019-485b-a30c-236918516958

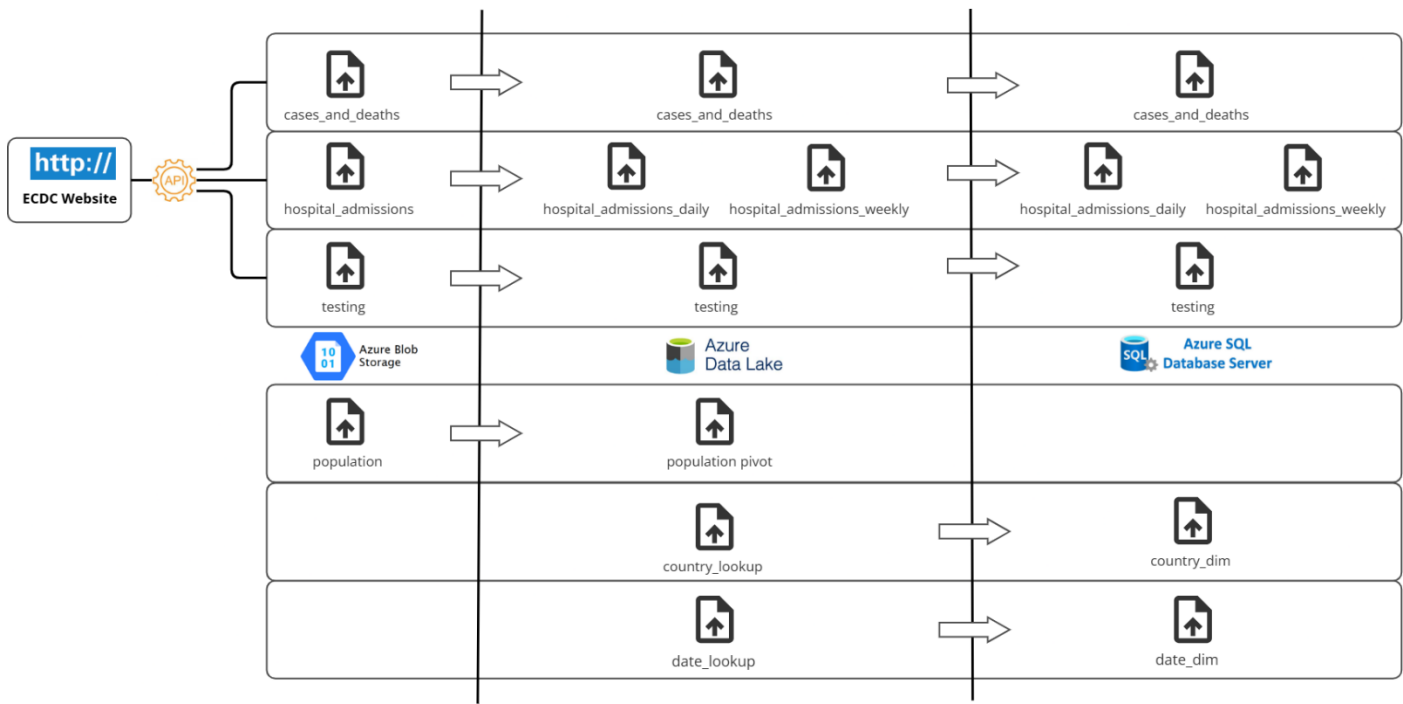
All status

Showing 1 - 6 of 6 items

Activity name	Activity type	Run start ↑↓	Duration	Status	Integration runtime	User properties	Error	Run ID
Copy ECDC Data	Copy	11/30/20, 2:45:46 PM	00:00:20	Failed	DefaultIntegrationRuntime (UK South)			a605509d-4de7-4d8e-b5ee-d499489ea896
Copy ECDC Data	Copy	11/30/20, 2:45:46 PM	00:00:20	Failed	DefaultIntegrationRuntime (UK South)			8d3df5fb-a992-4501-8d9e-617a01f8ad7c
Copy ECDC Data	Copy	11/30/20, 2:45:46 PM	00:00:19	Failed	DefaultIntegrationRuntime (UK South)			dffe21ae-eeb5-415b-bb6e-1f0e357308b7
Copy ECDC Data	Copy	11/30/20, 2:45:46 PM	00:00:18	Failed	DefaultIntegrationRuntime (UK South)			c3f6c1bd-df12-4405-98ce-62579e45759e
Execute Copy For Every Record	ForEach	11/30/20, 2:45:45 PM	00:00:23	Failed	Unknown			0f8a1a77-c2fa-4c31-9bf5-f0eee27300e7
Lookup ECDC File List	Lookup	11/30/20, 2:45:42 PM	00:00:03	Succeeded	DefaultIntegrationRuntime (UK South)			81d130bf-3d34-47d1-a4ae-bbe91df5f1c3

Azure Log Analytics

From Log analytics service, we can send all or selected logging details of selected service (in this case, Azure Data Factory) to Azure logs.



Data Sources

The project is about analyzing Covid19 dataset.

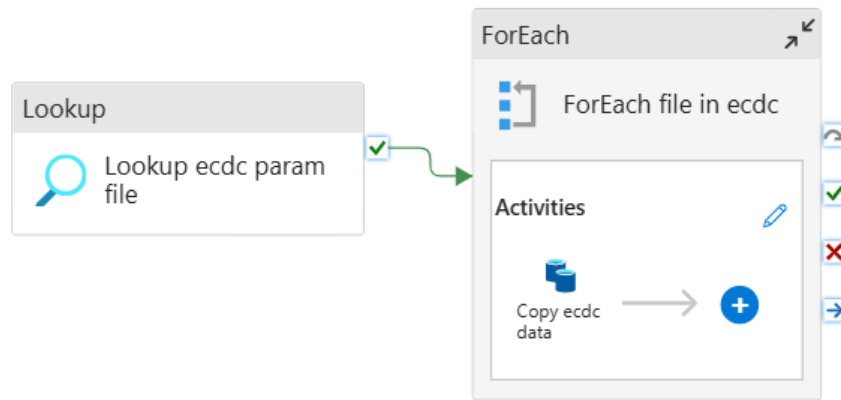
There were 4 files to ingest: 3 from ECDC website & 1 from Azure Blob Storage

- ECDC files - cases_and_deaths file (with data on number of cases & deaths across countries, timeframes and different age groups), hospital_admissions file (with data on number of hospital admissions for bed and ICU across countries, timeframes and different age groups), testing file (with data on number of tests conducted, testing rate and positivity rate across countries and timeframes)
- Azure blob file - population file (with population numbers across countries and timeframes)

Ingestion pipeline for files from ECDC

Used parameters/variables and metadata file to ingest all 3 files from ecdc website to Azure Data Lake Storage raw container. The source details, source and target file names were parameterized and fetched from the reference metadata file.

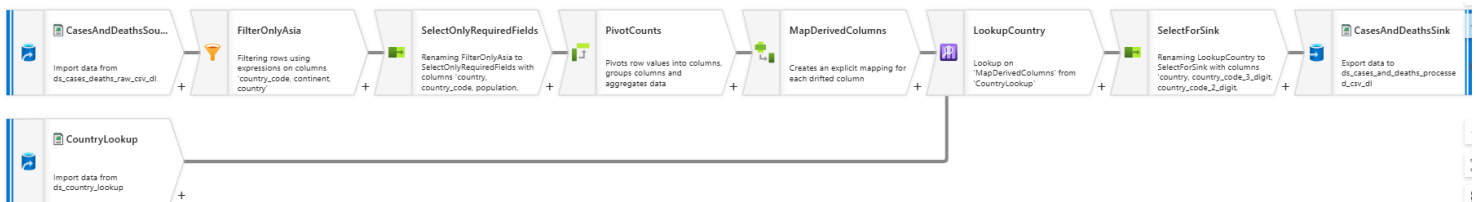
Transformations/Activities Used: Lookup, Copy, ForEach



Processing of Cases & Deaths File

Ingest from ecdc website via Azure Data Factory (ADF) pipeline and store in raw container in Azure Data Lake Storage (ADLS) -> transform using data flows in ADF pipeline and store in processed container in Azure Data Lake Storage (ADLS) -> load processed file into Azure SQL Database.

Transformations/Activities Used: Source, Filter, Select, Pivot, Derived Column, Lookup, Sink

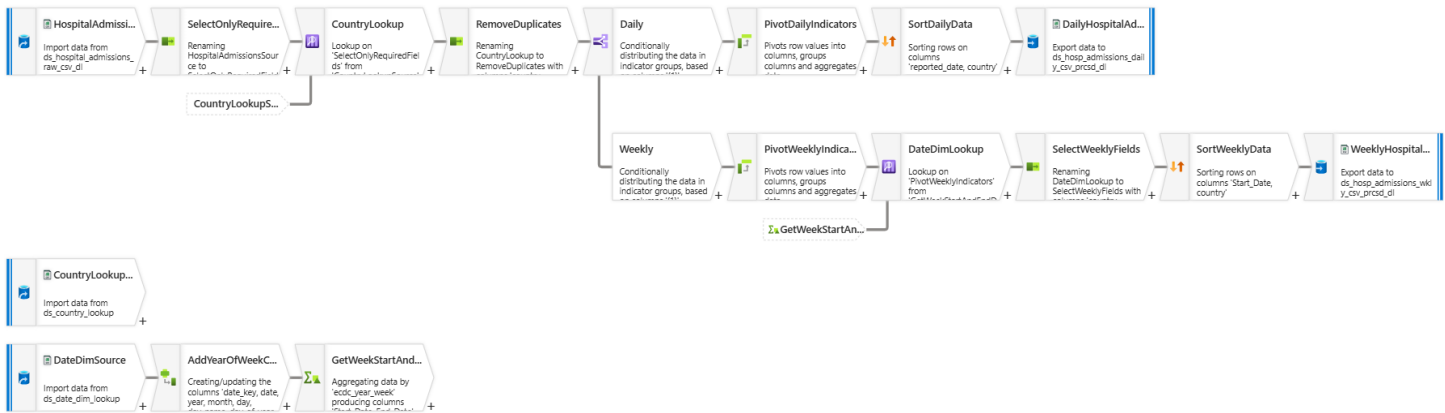


Processing of Hospital Admissions File

Ingest from ecdc website via Azure Data Factory (ADF) pipeline and store in raw container in Azure Data Lake Storage (ADLS) -> transform using data flows in ADF pipeline and store in processed container in Azure Data Lake Storage (ADLS) -> load processed file into Azure SQL Database

In the transform section, we split the files into daily and weekly aggregated data. From a single pipeline, two target files were created - hospital_admissions_daily & hospital_admissions_weekly.

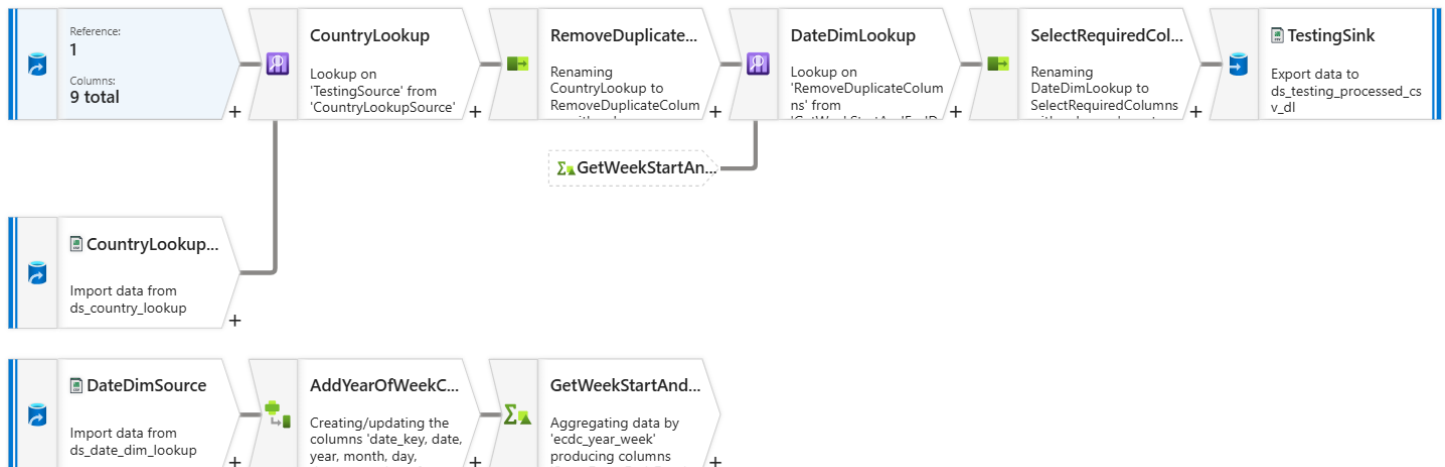
Transformations/Activities Used: Source, Select, Lookup, Conditional Split, Pivot, Sort, Aggregate, Sink



Processing of testing File

Ingest from ecdc website via Azure Data Factory (ADF) pipeline and store in raw container in Azure Data Lake Storage (ADLS) -> transform using data flows in ADF pipeline and store in processed container in Azure Data Lake Storage (ADLS) -> load processed file into Azure SQL Database

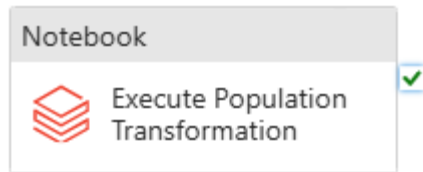
Transformations/Activities Used: Source, Lookup, Select, Aggregate, Sink



Processing of Population File

Ingest from Azure Blob Storage via Azure Data Factory (ADF) pipeline and store in raw container in Azure Data Lake Storage (ADLS) -> transform using Azure Databricks activity in ADF pipeline and store in processed container in Azure Data Lake Storage (ADLS) -> load processed file into Azure SQL Database

Transformations/Activities Used: Databricks



Other Related tasks

- Created linked services & datasets in ADF for all data objects.
- Spun up Databricks cluster, created access token, did App Registration in ADLS, mounted ADLS containers in Databricks and enabled access via token in ADF linked service.
- Spun up Azure SQL Database, added SQL authentication by creating new user in master and enabling the user in the covid-db.
- Created tables in Azure SQL Database and loaded them.

ADF Pipelines

Factory Resources

⌵ ⏪

Filter resources by name

+

▲ Pipelines	17
▲ Covid19 Project	17
▶ Execute	1
▶ Extract	5
▶ Load	6
▶ Misc	1
▶ Transform	4
▶ Change Data Capture (preview)	0
▲ Datasets	22
▲ Covid19 Project	22
▶ loaded	6
▶ lookup	...
▶ metadata	2
▶ processed	5
▶ raw	7
▲ Data flows	3
▲ Covid19 Project	3
df_transform_cases_deaths	
df_transform_hospital_admissions	
ds_transform_testing	

▲ Pipelines	...
▲ Covid19 Project	17
▲ Execute	1
pl_execute_population_pipelines	
▲ Extract	5
pl_ingest_cases_deaths_data	
pl_ingest_ecdc_data_metadata_driven	
pl_ingest_hospitalisation_data_using_variables	
pl_ingest_population_data	
pl_ingest_testing_data_with_runtime_parameters	
▲ Load	...
pl_date_dim_data	
pl_load_cases_deaths_data	
pl_load_country_dim_data	
pl_load_hosp_admissions_dly_data	
pl_load_hosp_admissions_wkly_data	
pl_load_testing_data	
▶ Misc	1
▶ Transform	4

Factory Resources

⌵ ⏪

🔍 Filter resources by name +

▸ Pipelines 17

▸ Change Data Capture (preview) 0

▴ Datasets 22

▴ 📁 Covid19 Project 22

▴ 📁 loaded 6

📊 ds_cases_deaths_tbl_sqldb

📊 ds_country_dim_sqldb

📊 ds_date_dim_sqldb

📊 ds_hosp_admissions_dly_tbl_sqldb

📊 ds_hosp_admissions_wkly_tbl_sqldb

📊 ds_testing_tbl_sqldb

▴ 📁 lookup 2

📊 ds_country_lookup

📊 ds_date_dim_lookup

▴ 📁 metadata ...

📊 ds_ecdc_2_file_list

📊 ds_ecdc_full_list

▸ 📁 processed 5

▸ 📁 raw 7

▴ Pipelines 17

▴ 📁 Covid19 Project 17

▸ 📁 Execute 1

▸ 📁 Extract 5

▸ 📁 Load 6

▴ 📁 Misc ...

🔗 pl_test_lkp_foreach

▴ 📁 Transform ...

🔗 pl_transform_cases_deaths_data

🔗 pl_transform_hosp_admissions_data

🔗 pl_transform_population_data

🔗 pl_transform_testing_data

Factory Resources



Filter resources by name

+

▲

Datasets

...

- ▲

📁

Covid19 Project

22
- 📁

loaded

6
- 📁

lookup

2
- 📁

metadata

2
- ▲

📁

processed

5

- 📄

ds_cases_and_deaths_processed_csv_dl
- 📄

ds_cases_deaths_raw_csv_http
- 📄

ds_hosp_admissions_daily_csv_prcsd_dl
- 📄

ds_hosp_admissions_wkly_csv_prcsd_dl
- 📄

ds_testing_processed_csv_dl

▲

📁

raw

...

- 📄

ds_cases_deaths_raw_csv_dl
- 📄

ds_ecdc_data_raw_dl
- 📄

ds_ecdc_data_raw_http
- 📄

ds_hospital_admissions_raw_csv_dl
- 📄

ds_population_raw_gz
- 📄

ds_population_raw_tsv
- 📄

ds_testing_raw_csv_dl

Factory Resources



Filter resources by name

+

▸

Pipelines

17

▸

Change Data Capture (preview)

0

▸

Datasets

...

▲

Data flows

3

▲

📁

Covid19 Project

3

- 🔄

df_transform_cases_deaths
- 🔄

df_transform_hospital_admissions
- 🔄

ds_transform_testing

▸

Power Query

0